

BHTTP — Binary HTTP-style Protocol

BHTTP is a binary protocol for serving files over one persistent TCP connection. Frames are length-prefixed. All integers use big-endian byte order. **MUST**, **MUST NOT**, and **MAY** have their RFC 2119 meanings.

1. Connection

BHTTP runs over TCP. The client opens one connection and may send multiple requests on it. The server keeps the connection open and processes requests in order until the client closes it.

2. Frames

Each frame has an 8-byte header: **Length (24 bits)**, **Type (8 bits)**, **Flags (8 bits)**, **Request ID (24 bits)**. Length is the payload size and excludes the header. The maximum payload is 16,777,215 bytes. The Request ID is chosen by the client and is copied to all response frames for that request. Undefined flags **MUST** be zero when sent and ignored when received.

0x01 REQUEST is sent by the client. Its payload is `method:u8 | path_len:u16 | path | headers`. Method **1** means GET. The path **MUST** be UTF-8 and start with `/`.

0x02 RESPONSE is sent by the server before DATA. Its payload is `status:u16 | headers`. Status uses HTTP codes; BHTTP defines 200, 400, and 404.

0x03 DATA contains response bytes. Flag **0x01** is **END_STREAM** and **MUST** be set on the final DATA frame. A response with no body uses one empty DATA frame with this flag.

Unknown frame types **MUST** be read using Length, discarded, and ignored.

3. Headers

A header block starts with `count:u8`, followed by that many entries.

An indexed entry uses `0x81–0x8A`, followed by `value_len:u16` and the UTF-8 value. The indexes are: **1 host**, **2 user-agent**, **3 accept**, **4 content-length**, **5 content-type**, **6 server**, **7 date**, **8 last-modified**, **9 etag**, **10 connection**.

A literal entry uses `0x00 | name_len:u8 | name | value_len:u16 | value`. Names **MUST** be non-empty lowercase ASCII and values **MUST** be valid UTF-8. Any other marker is invalid.

4. Server

The server resolves the request path relative to its document root. `/` and directories map to `index.html`. The resolved real path MUST remain inside the document root.

For an existing file, the server sends `RESPONSE 200` with `server`, `content-type`, and `content-length`, followed by one or more DATA frames ending in `END_STREAM`. Files are split into 64 KiB DATA frames.

A missing file or path outside the root returns `RESPONSE 404` followed by an empty DATA frame with `END_STREAM`.

A malformed REQUEST returns `RESPONSE 400` followed by an empty DATA frame with `END_STREAM`. Malformed input includes truncated fields, invalid UTF-8, invalid markers, unsupported methods, invalid paths, and trailing bytes. The connection remains open if the REQUEST frame header was successfully read.

Inbound frames are limited to 64 KiB. If Length exceeds this limit, the server sends `400` for that Request ID and closes the connection without reading the payload. Non-REQUEST frames are skipped. If a connection ends in the middle of a frame, it is dropped without a response.

5. Client

The client sends a REQUEST and reads frames until the matching DATA frame has `END_STREAM`; unknown frames are skipped. The response body is written to stdout.

Exit status is `0` for responses below 400, `22` for 4xx/5xx responses, and `1` for network or protocol errors. With `-v`, sent frames are hexdumped with `>` and received frames with `<`.

6. Example

For `./bcurl -v localhost:9000/index.html`, the request uses Request ID 1, method GET, path `/index.html`, and the headers `host`, `user-agent`, and `accept`. The server replies with `200`, the `server`, `content-type`, and `content-length` headers, followed by DATA frames containing the file. The final DATA frame has `END_STREAM`.

A minimal exchange is therefore:

```
REQUEST → RESPONSE 200 → DATA → ... → DATA END_STREAM
```

For an error:

```
REQUEST → RESPONSE 404 → empty DATA END_STREAM
```

